Advanced C Course

Code Quality applied to C

Pascal Rothnemer
Spring 2026, EPITA

Course Highlights

- C /C++ Language Fundamentals
  - History, principles, features
  - Pros & Cons, Applications
- Code Quality in C/C++
  - Why - User view, Owner view
  - Assessing Code Quality
  - Best Practices – how to write good code

```c
#include <stdio.h>

int main() {
  printf("Hello World\n");
  return 0;
}
```

Course Program - Sessions 1-7

1) Hello, World !

- Computer Fundamentals
- C/C++ presentation and history
- IDE setup

2) Types, objects, basic pointers

- Developing on VS 2022
- Code quality : Useability, Documentation

3) C++, OOP, Classes

4) Functions, libraries

5) Arrays, Vectors

- Debugging on VS 2022
- Code quality : Robustness, exception handling

6) Loops

- metrics on VS 2022
- Code quality : Performance, metrics , optimization

7) Projects Presentations & Evaluation

Session 1 – Let's Start !

- *«Programming in C vs most other languages is like learning to drive a manual car vs. an automatic. »*

- *« Understanding C will give you a better appreciation of Computer Architecture & Operating Systems.»*

Computer Fundamentals

- **Base 2 - binary** numeral system
- **Base 16 - hexadecimal** numeral system
- **BIT – Binary Digit –** two possible values **: 0 and 1**
- **Byte =** a sequence of **8 bits**
- **Word =** 2 bytes **= 16 bits**
- **Long Word =** 4 bytes **= 32 bits**

- **CPU = Machine** able to **execute** (a set of) **Instructions**
- **Computer memory :** stores information, data and programs, for use in the computer.
- **Motherboard :** main printed circuit board (PCB) in general-purpose computers (PC...)
- **Operating System :** software that manages a computer's hardware and applications by allocating resources, including memory, CPU, input/output devices and file storage.

Computer Fundamentals

Base 10 : 10 figures used to describe any number

- 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
- Number ZERO is noted « 0 »
- Number ONE is noted « 1 »
- [...]
- Number NINE is noted « 9 »
- Number TEN is noted 10
- Number ELEVEN is noted 11
- ETC...

Computer Fundamentals

Base 2 : Only 2 figures are used to describe any number

- 0, 1
- Number ZERO is noted « 0 »
- Number ONE is noted « 1 »
- Number TWO is noted « 10 »
- Number THREE is noted 11
- Number FOUR is noted 100
- ETC...

Computer Fundamentals

Base 16 : 16 figures used to describe any number

- 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F
- Number ZERO is noted « 0 »
- Number ONE is noted « 1 »
- [...]
- Number NINE is noted « 9 »
- Number TEN is noted « A »
- Number FIFTEEN is noted « F »
- Number SIXTEEN is noted « 10 »
- ETC...

Computer Fundamentals

Base 2 : Exercise

Number	Base 10 representation	Base 2 representation
Two	2	10
Fifteen	15	?
Sixteen	16	?
Twenty one	21	?

Computer Fundamentals

Base 2 : Exercise

Number	Base 10 representation	Base 2 representation
Two	2	10
Fifteen	15	1111
Sixteen	16	10000
Twenty one	21	10101

Computer Fundamentals

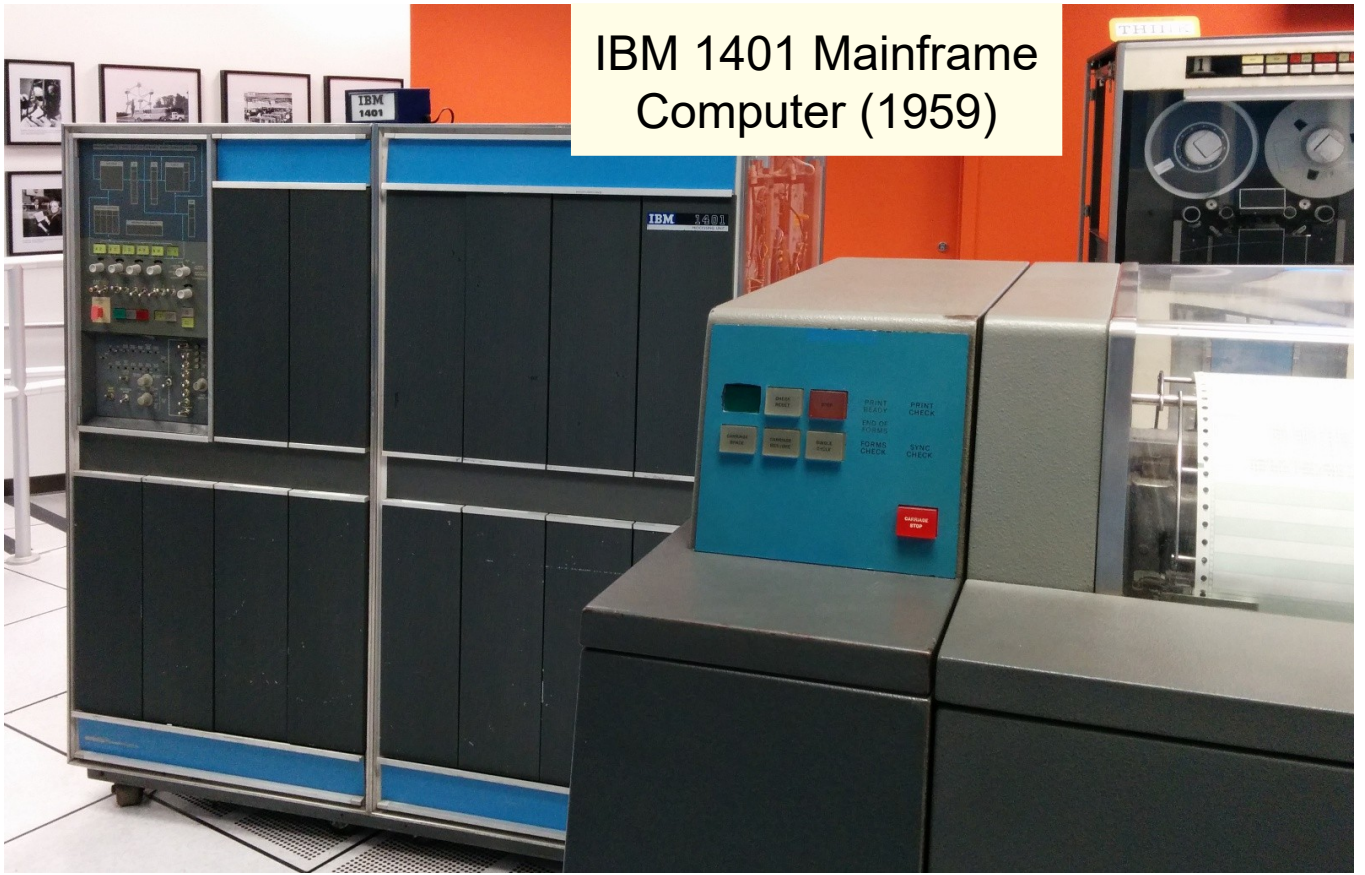
Base 10, Base 2, Base 16

Number	Base 10 representation	Base 2 representation	Base 16 representation
Two	2	10	2
Five	5	101	5
Fifteen	15	1111	F
Sixteen	16	10000	10

Computer Fundamentals

Why do we use Binary System in Computer Sciences ?

- **Efficient way to represent values in the form of electrical signals (ON-OFF, Current/No Current)**

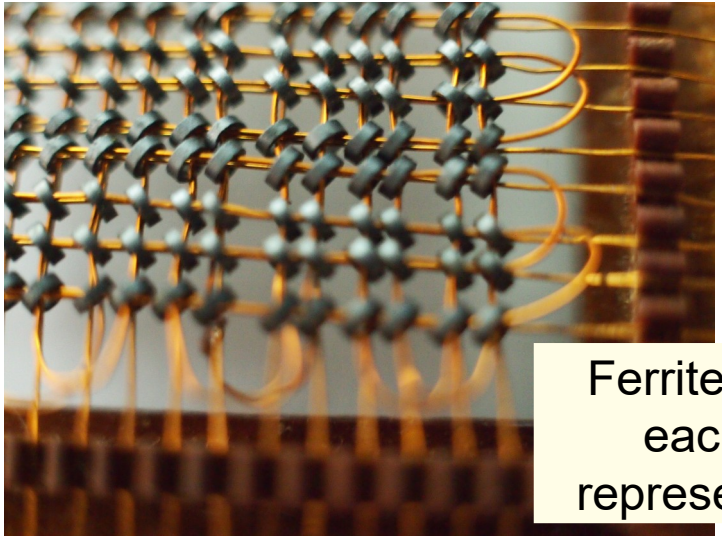


IBM 1401 Mainframe Computer (1959)

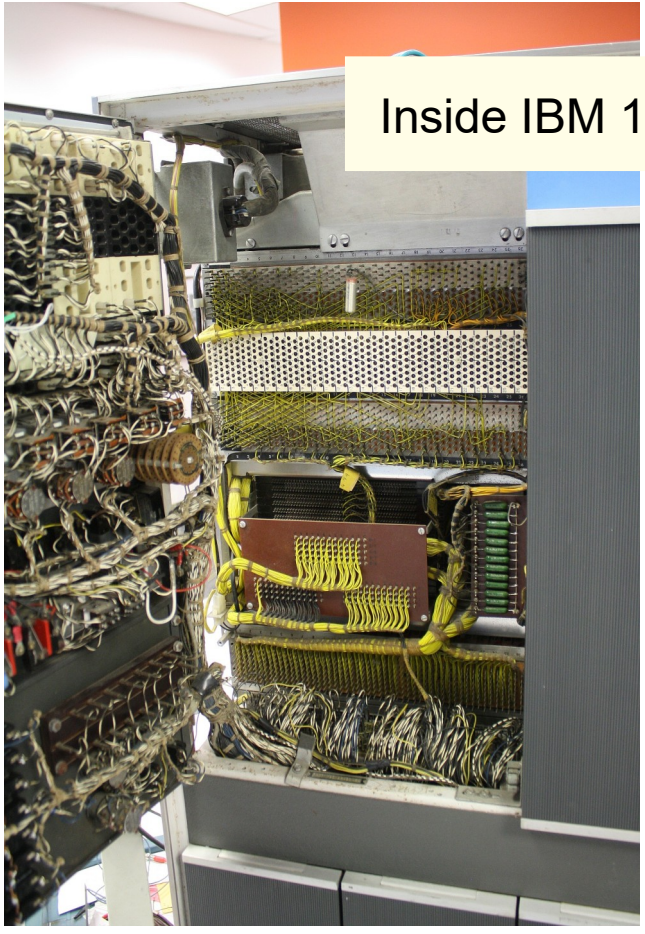
Computer Fundamentals

Early Memory physics

- **Based on Ferrite Cores through which current**
- **Passes (« 1 » value) - doesn’t pass (« 0 » value)**



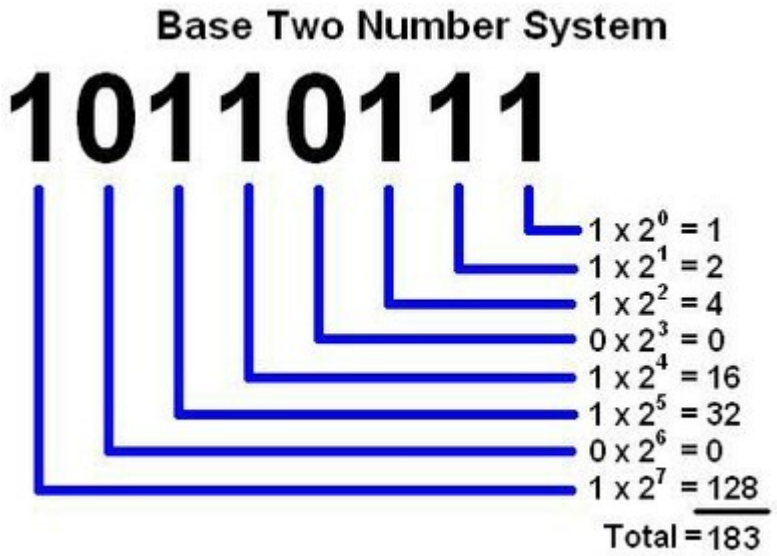
Ferrite cores – each one represents 1 bit



Inside IBM 1401

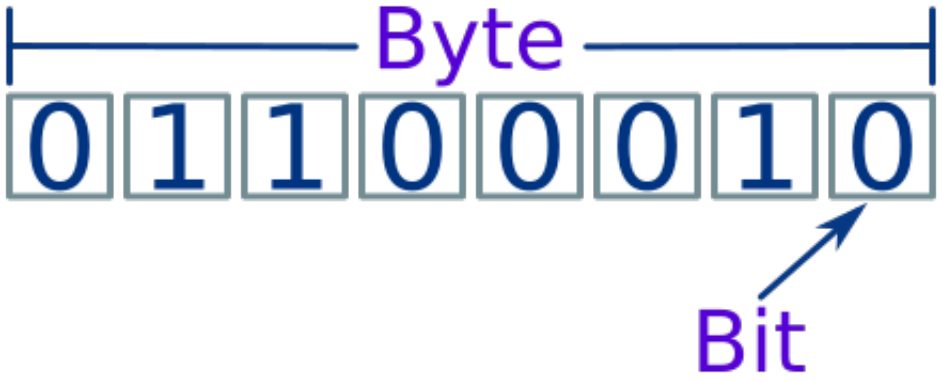
Computer Fundamentals

base-2 or **binary numeral system**



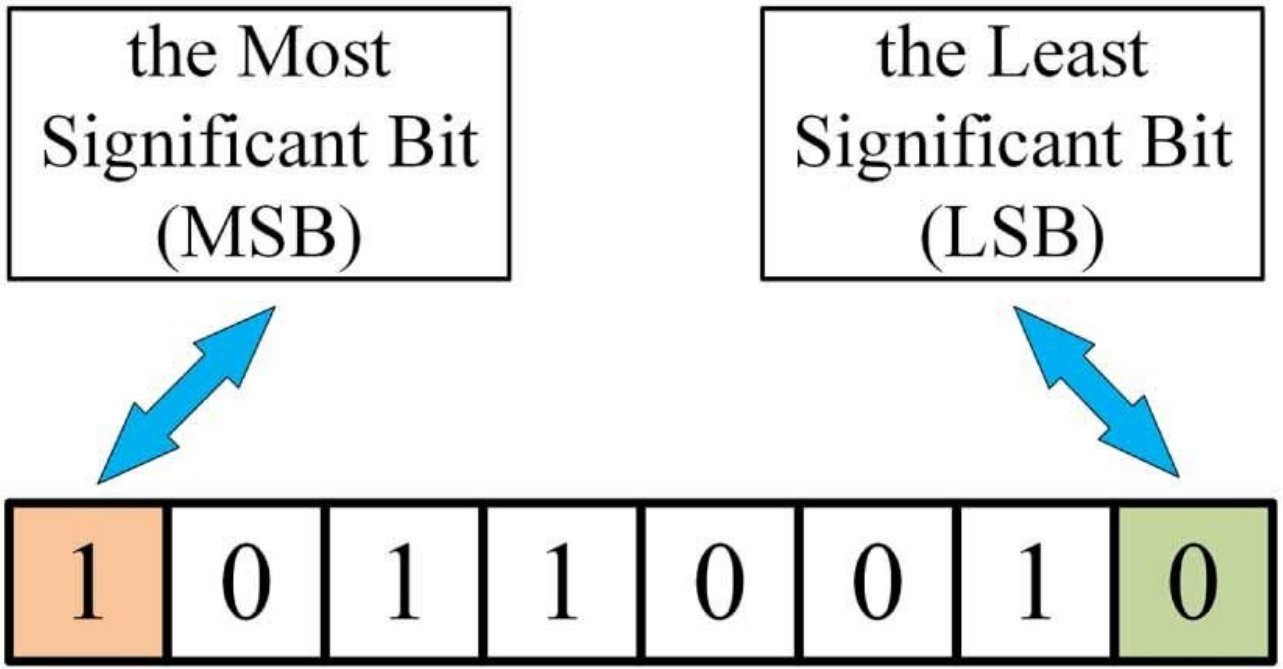
Computer Fundamentals

- **base-2** or **binary numeral system**
- **BIT – Binary Digit – two possible values : 0 and 1**
- **Byte – a sequence of 8 bits**



Computer Fundamentals

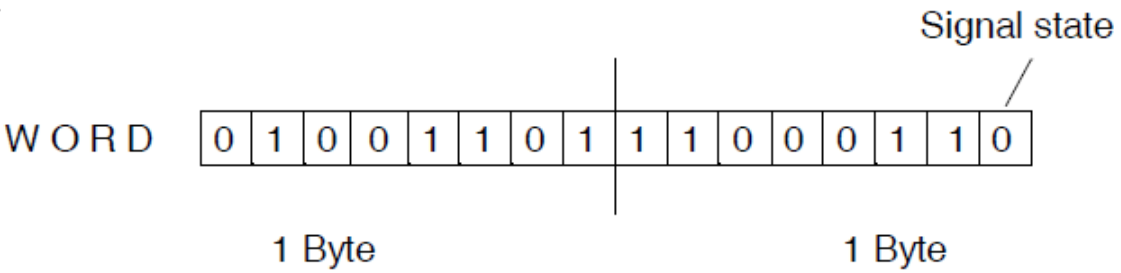
- **MSB - LSB**

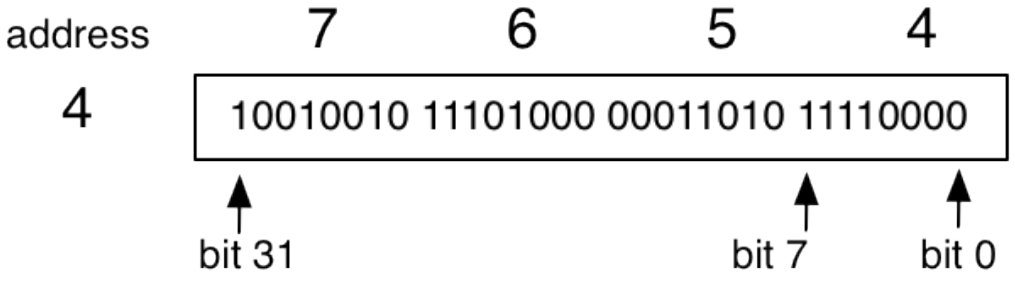


Computer Fundamentals

- **Word – 2 bytes = 16 bits**

Long Word = 4 bytes = 32 bits



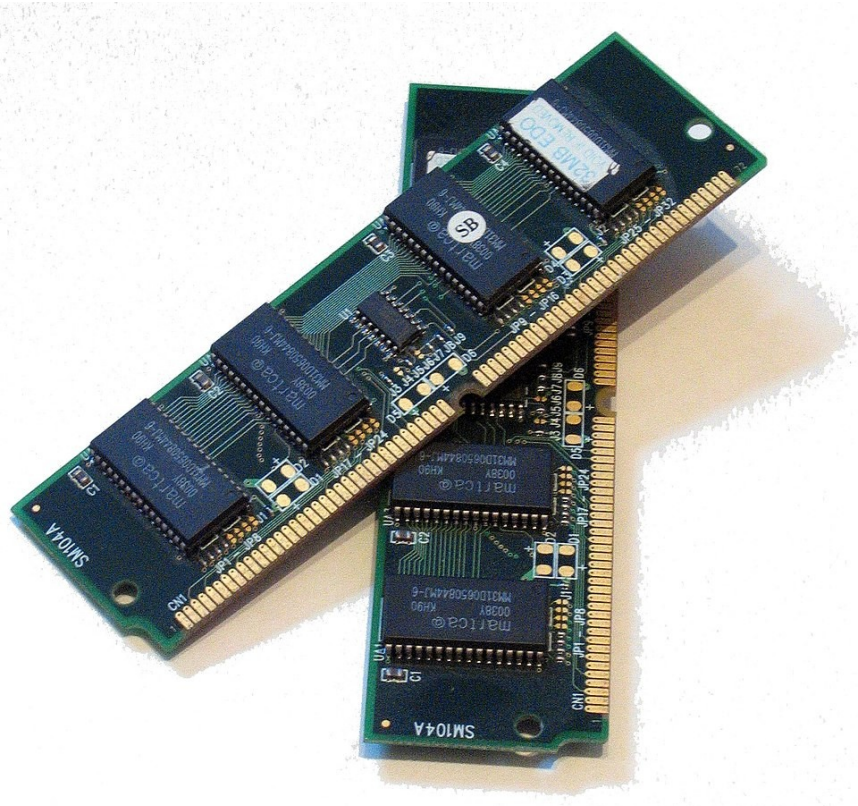


Computer Fundamentals

- **CPU – Central Processing Unit**
- the primary processor in a given computer. Its electronic circuitry **executes instructions** of a computer program, such as arithmetic, logic, controlling, and input/output (I/O) operations.
- Interacts with external components such as main memory, I/O circuitry, specialized coprocessors (graphics processing units - GPUs).

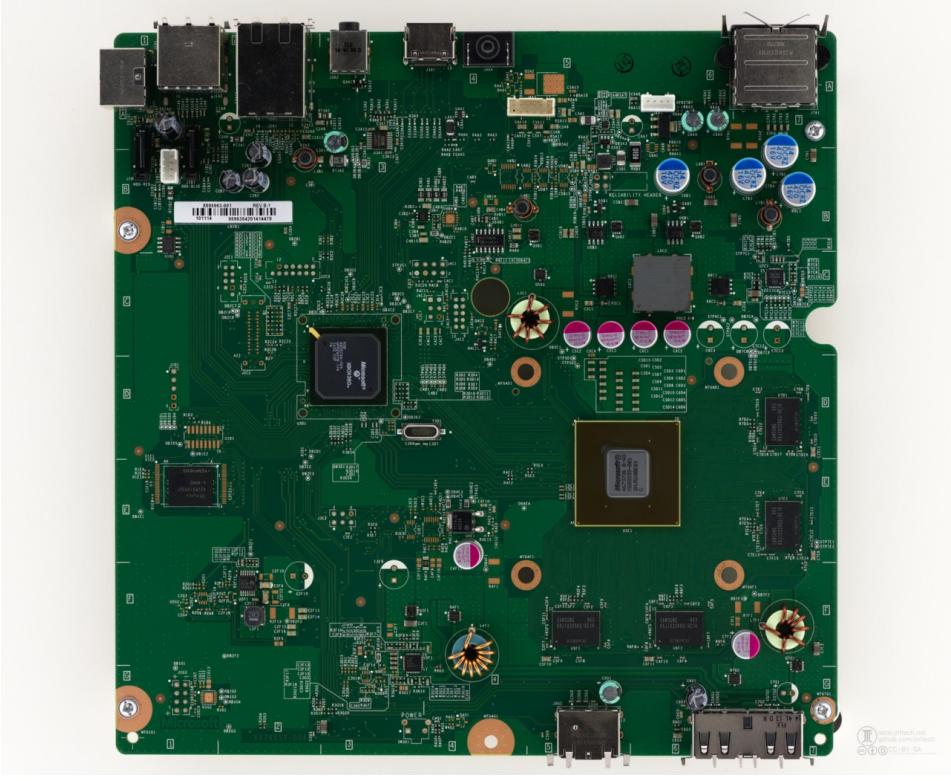
Computer Fundamentals

- **Computer memory**
- stores information, such as data and programs, for immediate use in the computer
- Program instructions and data are located in computer memory, they are fetched and stored during execution.
- Often referred to as RAM, i.e. « random-access memory »



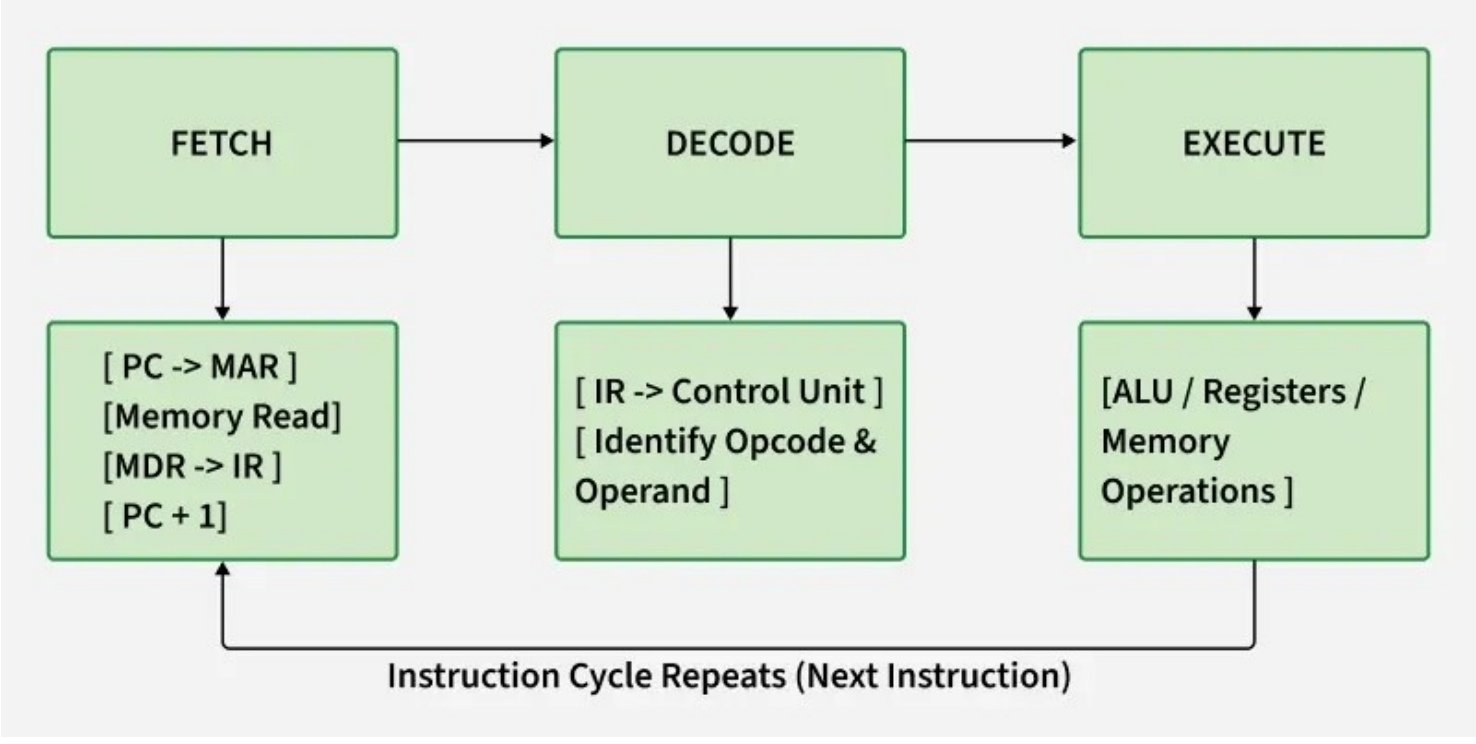
Computer Fundamentals

- **Motherboard**
- main printed circuit board (PCB) in general-purpose computers (PC...)
- Holds and allows communication between crucial electronic components : **CPU, memory**
- provides connectors for peripherals.



Computer Fundamentals

Typical execution of an instruction by a microprocessor



Computer Fundamentals

Instruction Set Architecture (ISA)

The formal specification that defines all the instructions a CPU can understand and execute. It includes the opcodes, instruction formats, available registers, memory access modes, and control behavior. Common ISAs include **x86, ARM, RISC-V,** and **MIPS**. The ISA forms the boundary between hardware and software: compilers, operating systems, and binary programs target a specific ISA, while CPU designers implement its functionality in hardware.

Computer Fundamentals

Opcode example : MOV

MOV AL, 34h - will move hexadecimal value 0x34 inot register AL

The opcode is the MOV instruction. The other parts are the 'operands'.

opcode for MOV is 100010.

Operands are manipulated by the opcode. In this example, the operands are the register named AL and the value 34 hex.

Computer Fundamentals

Machine Instructions – example of Intel 8086 Microprocessor

Data transfer instructions- move, load exchange, input, output.

- MOV: Move byte or word to register or memory .
- IN, OUT: Input byte or word from port, output word to port.
- LEA: Load effective address
- LDS, LES Load pointer using data segment, extra segment .
- PUSH, POP: Push word onto stack, pop word off stack.
- XCHG: Exchange byte or word.
- XLAT: Translate byte using look-up table.

Computer Fundamentals

Machine Instructions – example of Intel 8086 Microprocessor

Arithmetic instructions - add, subtract, increment, decrement, convert byte/word and compare.

- ADD, SUB: Add, subtract byte or word
- ADC, SBB: Add, subtract byte or word and carry (borrow).
- INC, DEC: Increment, decrement byte or word.
- NEG: Negate byte or word (two's complement).
- CMP: Compare byte or word (subtract without storing).
- MUL, DIV: Multiply, divide byte or word (unsigned).
- IMUL, IDIV: Integer multiply, divide byte or word (signed)
- CBW, CWD: Convert byte to word, word to double word
- AAA, AAS, AAM ,AAD: ASCII adjust for add, sub, mul, div .
- DAA, DAS: Decimal adjust for addition, subtraction (BCD numbers)

Computer Fundamentals

Machine Instructions – example of Intel 8086 Microprocessor

Logic instructions

- - AND, OR, exclusive OR, shift/rotate and test
- NOT: Logical NOT of byte or word (one's complement)
- AND: Logical AND of byte or word
- OR: Logical OR of byte or word.
- XOR: Logical exclusive-OR of byte or word
- TEST: Test byte or word (AND without storing).
- SHL, SHR: Logical Shift rotate instruction shift left, right byte or word? by 1or CL
- SAL, SAR: Arithmetic shift left, right byte or word? by 1 or CL
- ROL, ROR: Rotate left, right byte or word? by 1 or CL.
- RCL, RCR: Rotate left, right through carry byte or word? by 1 or CL.

Computer Fundamentals

Machine Instructions – example of Intel 8086 Microprocessor

String manipulation instruction - load, store, move, compare and scan for byte/word

- MOVS: Move byte or word string
- MOVSB, MOVSW: Move byte, word string.
- CMPS: Compare byte or word string.
- SCAS S: can byte or word string (comparing to A or AX)
- LODS, STOS: Load, store byte or word string to AL.

Computer Fundamentals

Machine Instructions – example of Intel 8086 Microprocessor

Control transfer instructions

- - conditional, unconditional, call subroutine and return from subroutine.
- JMP: Unconditional jump .it includes loop transfer and subroutine and interrupt instructions.
- JNZ: jump till the counter value decreases to zero. It runs the loop till the value stored in CX becomes zero

Computer Fundamentals

Machine Instructions – example of Intel 8086 Microprocessor

Loop control instructions

- LOOP: Loop unconditional, count in CX, short jump to target address.
- LOOPE (LOOPZ): Loop if equal (zero), count in CX, short jump to target address.
- LOOPNE (LOOPNZ): Loop if not equal (not zero), count in CX, short jump to target address.
- JCXZ: Jump if CX equals zero (used to skip code in loop).
- Subroutine and Interrupt instructions-
- CALL, RET: Call, return from procedure (inside or outside current segment).
- INT, INTO: Software interrupt, interrupt if overflow.IRET: Return from interrupt.

Computer Fundamentals

What does it takes for a computer to execute this simple instruction

int a=5, b=7, c;

c=a+b ;

- Fetch (read) value of memory location at adress « a » into register R0
- Fetch (read) value of memory location at address « b » into register R1
- Perform arithmetic addition of registers R0 & R1
- Stores result into register R2
- Move (write) value of R2 into memory location at address « c »

Computer Fundamentals

ADD R0, R1, R2

- In binary, this might translate to: **0001 0001 0010 0011**

opcodes:

0001: Add two values

0010: Move data from one register to another

0100: Load a value from memory

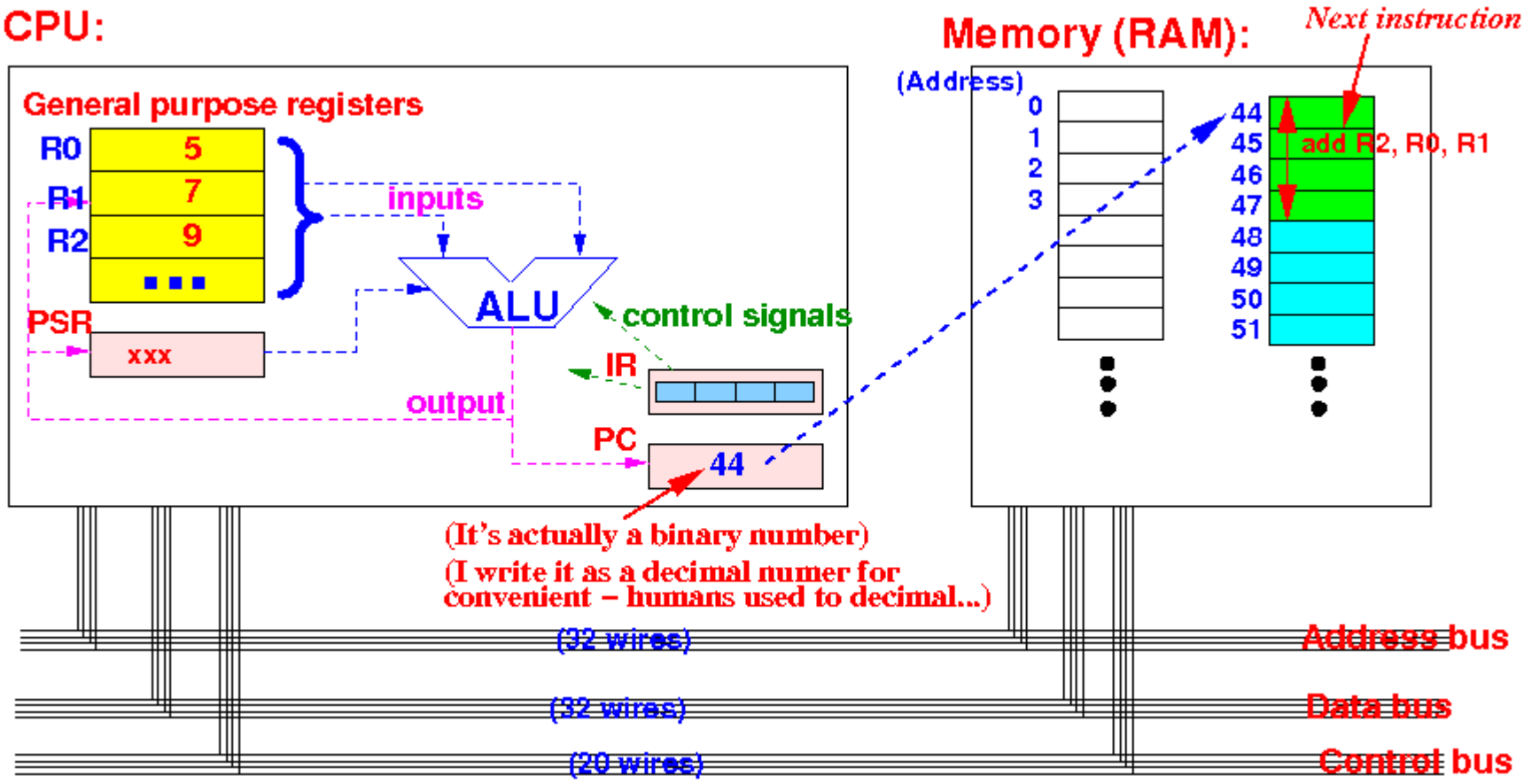
1000: Jump to a different memory location

Computer Fundamentals

Instruction

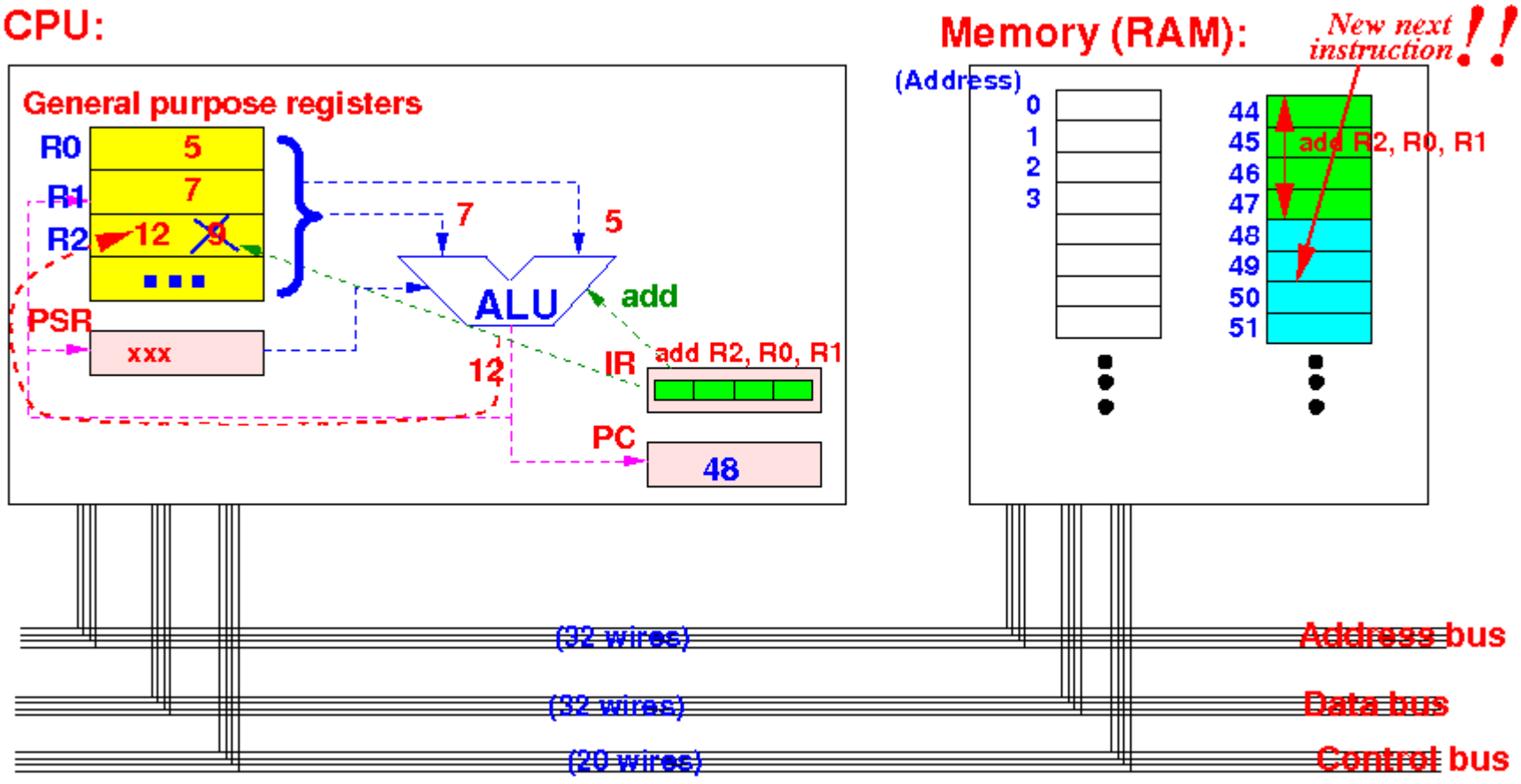
R2=R0+R1

BEFORE



Computer Fundamentals

Instruction
R2=R0+R1
AFTER



Computer Fundamentals

- **Operating System**
- software managing computer hardware and software resources
- provides common services for computer programs.
- schedules tasks for efficient use of the system
- Manages allocation of CPU time, mass storage, peripherals, other resources.
- acts as an intermediary between programs and hardware
- application code executed by the hardware frequently makes system calls to an OS function or is interrupted by it.
- Operating systems exist on any devices that contain a computer : cellular phones, video game consoles, web servers, supercomputers...

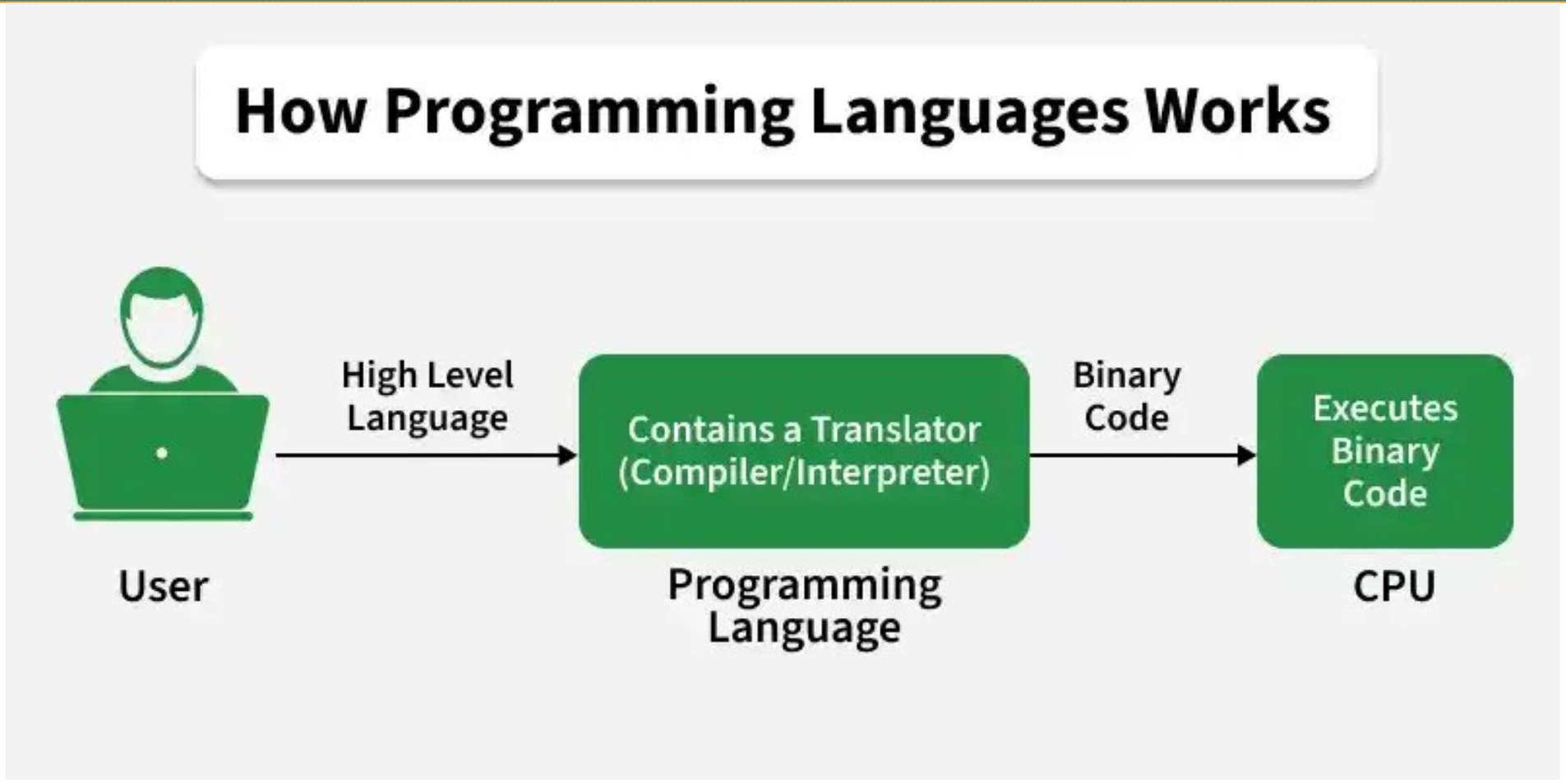
Programming Languages

Why do we need programming Languages

- A Computer does not speak any human langage
- It only executes statements from its instructions set
- Humans don’t know how to write Computer Instructions

=> We need an intermediate

Programming Languages



Programming Languages

Why do we need programming Languages

- There are two main approaches for implementing a programming langage
  - Compilation : programs are « compiled » ahead-of-time to generate machine code. **Example C/C++**
  - Interpretation : programs are interpreted line after line (i.e. checked and translated into machine code) and directly executed. **Example Python**
- Some implementations use hybrid approaches such as just-in-time compilation and bytecode interpreters. **Example JAVA**

Brief History (1) – Early days

- 1969 : **Unix** development in « Bell Labs », (Ken Thompson, Dennis Ritchie)...
- 1970 : «**B** » (**BCPL** = Basic Combined Programming Language).», to better write Unix. Ken Thompson and Dennis Ritchie
- 1972 : B → **C, based on B,** by Dennis Ritchie, to code Unix Utilities
- 1973 : Unix (V4) is fully (re)-written in C.
- 1978 : « **The C Programming Language** », by B. Kernighan and D. Ritchie, first specification of « **K&R C** » (or **C78)**

Brief History (2) – C++

- 1979 : **Bjarne Stroustrup** starts working on "C with Classes"...
  - Set out to enhance the C language with « Simula-like » features.
  - C chosen because general-purpose, fast, portable, and widely used.
- 1982 : Stroustrup starts to develop "**C++**" as a successor to C with Classes
- 1985 : first edition of « **The C++ Programming Language** »
  - became the definitive reference for the langage
  - first commercial implementation of C++ was released in October

C Main Features

- **Procedural Language** supporting **Structured Programming (**functions**)**
- **Compiled, Low-Level** → Fast & Efficient : Direct hardware & memory access
- **Statically typed** : the data type of a variable is known at compilation time.
- **General-Purpose** Language : can be used in any application / domain
- **built-in Operators**
- **Standard Libraries** with Rich Functions
- **Portable** : does not depend on hardware

C++ Main additional Features

- Object-Oriented Programming (OOP):
  - Abstraction
  - Encapsulation
  - Inheritance
  - Polymorphism
- Namespace, keywords, std libraries, syntax changes (simplification)...

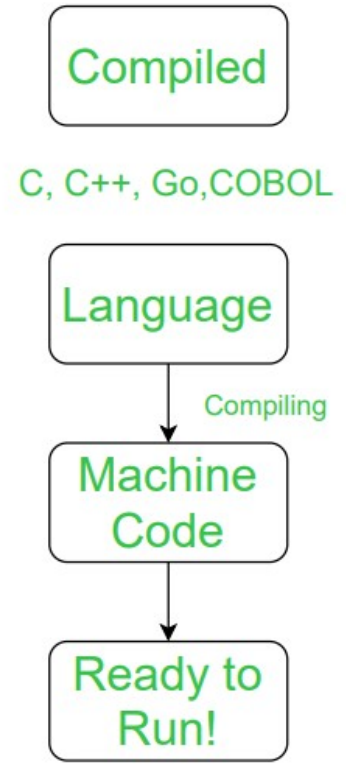
C/C++ Strengths

- **Compiled**: generated code is fast & optimized (executable)
- **Low Level :** directly manages computer resources (memory, CPU)
- **Performance :** because compiled & low level, C generates fast code
- **Portability**: compiles and runs on various machines with few changes.
- **Rich Library**: huge collection of built-in functions and libraries.
- **Security** : nearly impossible to crack (as opposed to, say, PYTHON :)

Compiled

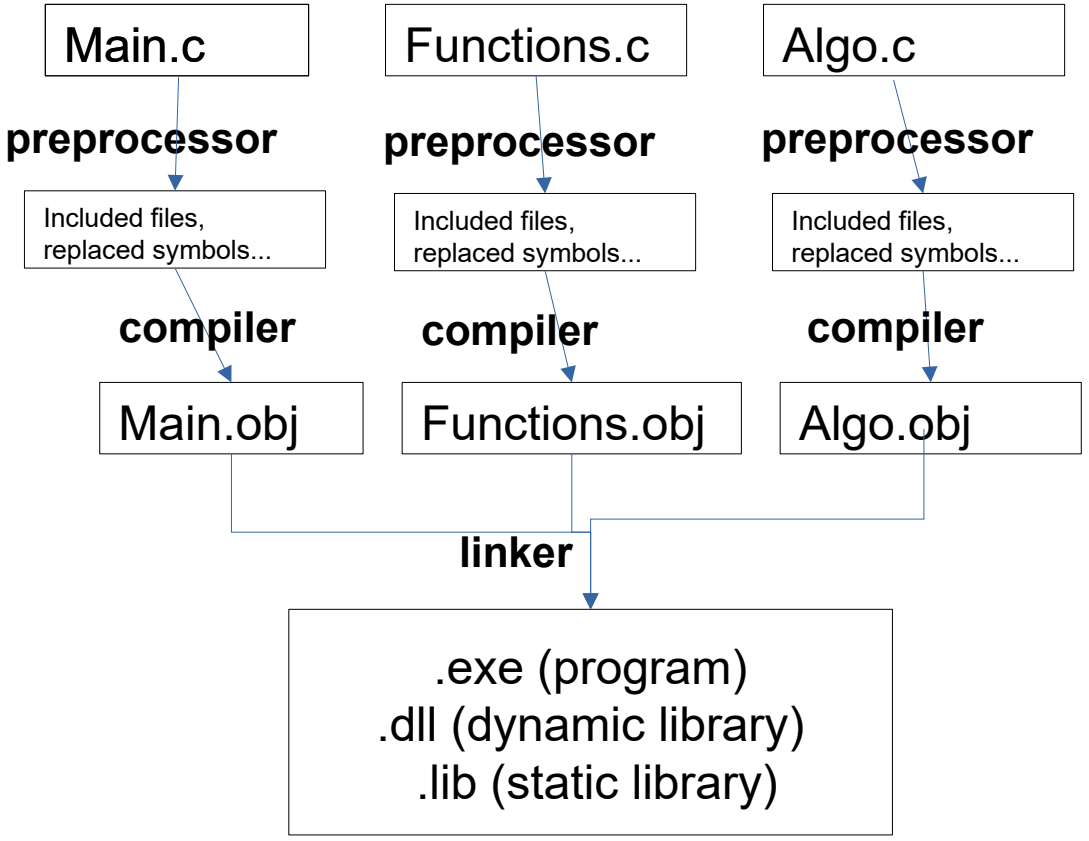
C/C++ : **Compiled**

The program is translated to machine code at once, then the machine code is run by the CPU



Code generation

Building a 3 files project in Visual Studio



Source files

Pre-processed Source files

object files : machin langage, but NOT executable

executable

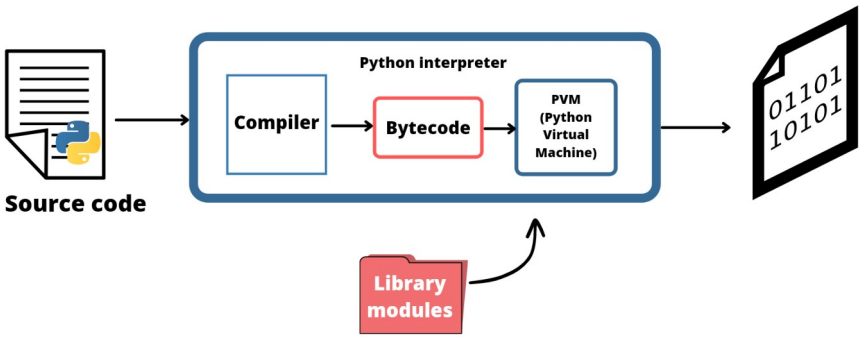
Exe vs Dll – BOTH are executables

- **Exe**: Program or Application
  - Must have a unique entry point : **Main**() function
  - Can/has to be started by the user within the OS
  - Owns its own memory space – runs as a process
- **Dll** : Dynamic Link Library
  - Does **NOT** define a **Main**() function
  - Can/has to be started by another Dll or an Exe within an existing process
  - Runs within the caller's memory space - process
  - Designed to better organize and duynamically optimize memory/resources

Interpreted

Python : **Interpreted**

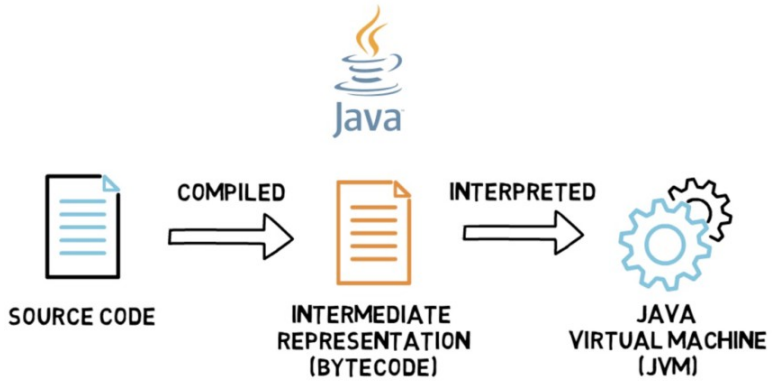
Code is read line-by-line and the machine instructions for that line are executed by the CPU.



« Compiled & Interpreted »

Java : **both compiled and interpreted**

Source code is first **compiled** into a binary byte-code. This byte-code runs on the Java Virtual Machine (JVM), which is usually a software-based interpreter



C Limitations

- Learning curve steeper than, say, Python.
- Low-level langage requires high-level (skilled!) programmer :-)
- Memory corruption is possible, due to programmer error.
- Few self-checks → protection is of programmer's responsibility
- Not object-oriented – (achieved by C++)

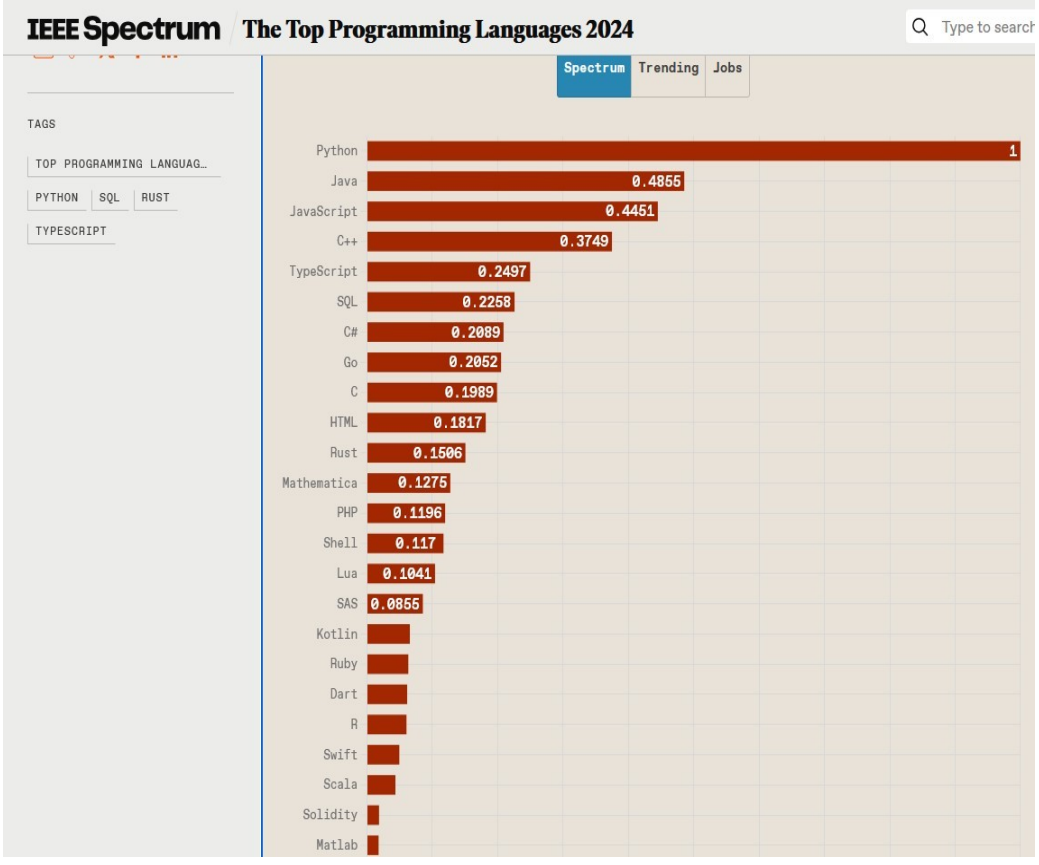
C Domains

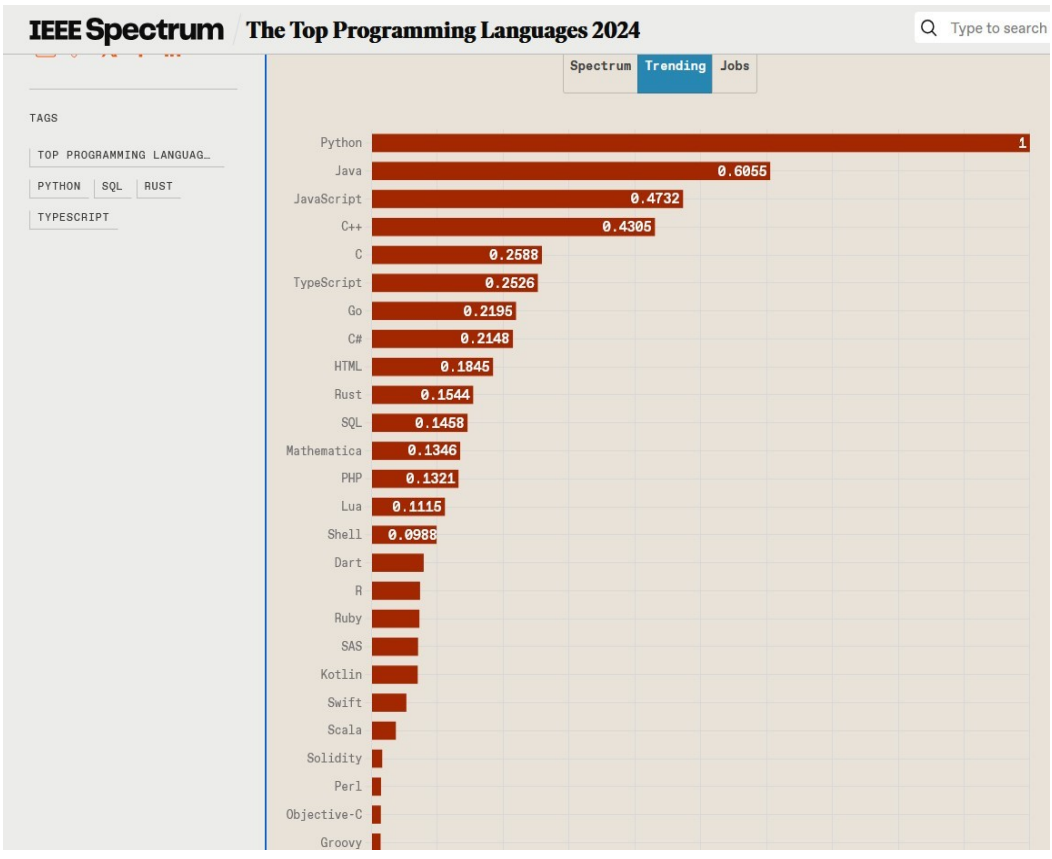
- **Operating Systems:** UNIX, LINUX, Androïd, MacOS...some parts of Windows
- **Compilers:** C itself , references of Python, Perl, Ruby, and PHP.
- **Embedded Systems:** Speed with limited CPU, memory
- **Technical & Scientific apps :** fast and efficent code, math libraries
- **Web Servers :** Apache HTTP and Nginx **(**link with Server OS, listen TCP ports for HTTP requests, serve web content, execute PHP, written in C**.**
- **Web BACK END** (algorithms, data access)
- **Database Management:** MySQL
- **Drivers, Utilities, Text Editors**

Out of C Scope

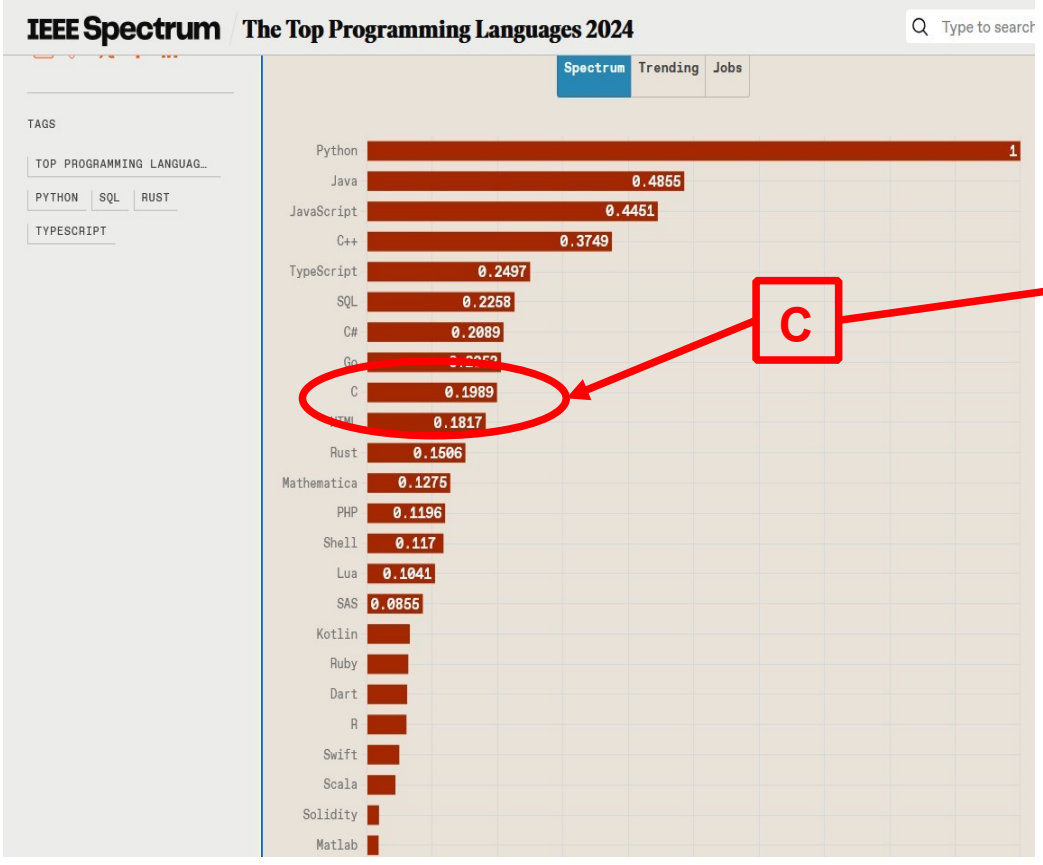
- Web Front End
- Not a language for dummies :-)
- « OOP » : Object-Oriented programming (C++)
- That’s it...but it is a lot ☺

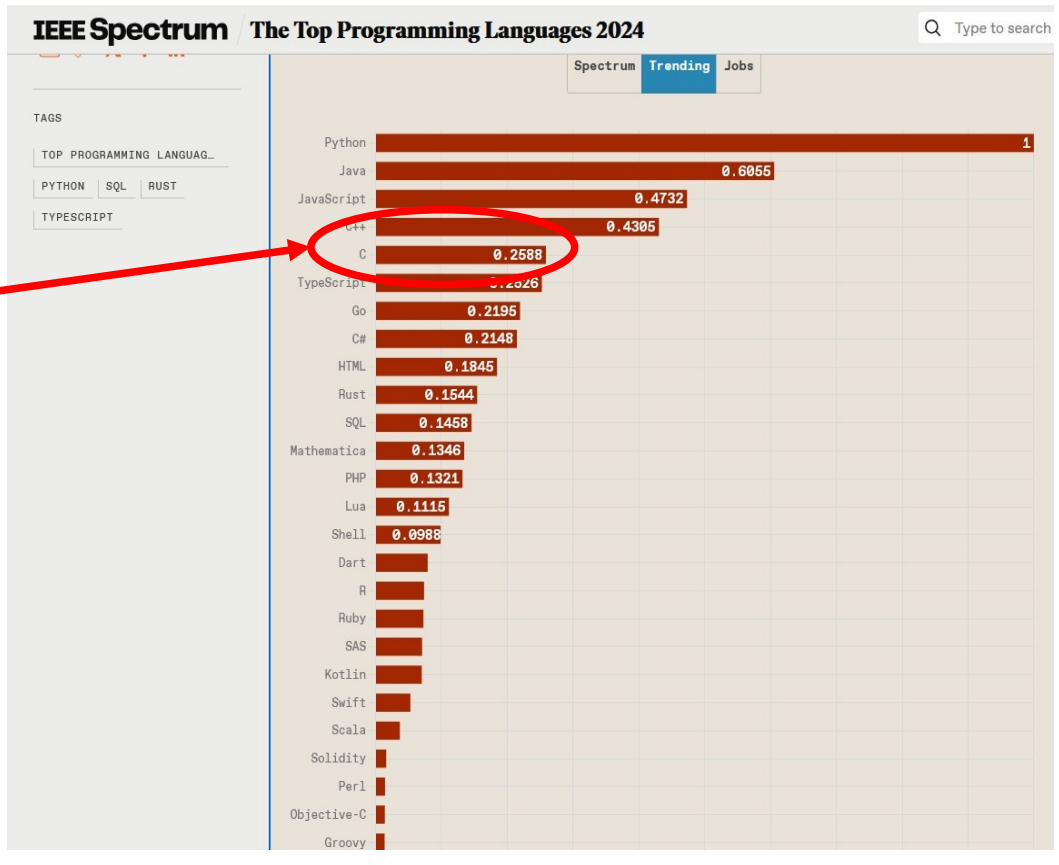
C Popularity Today



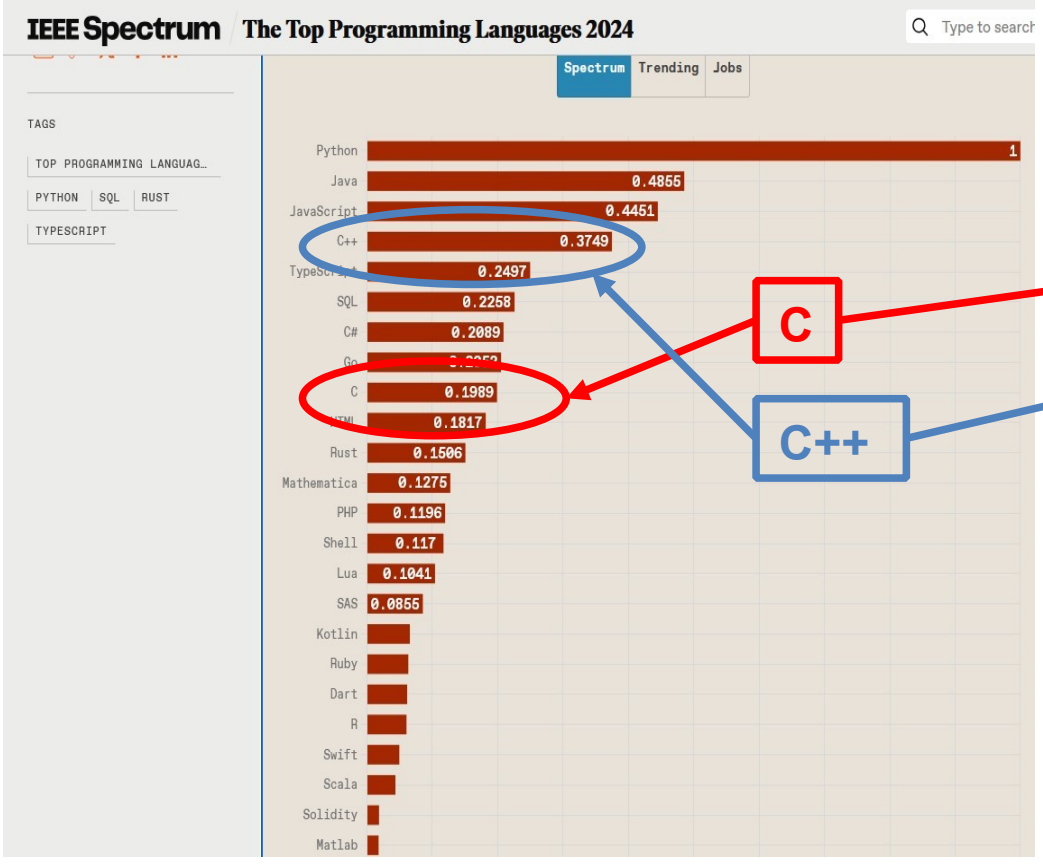


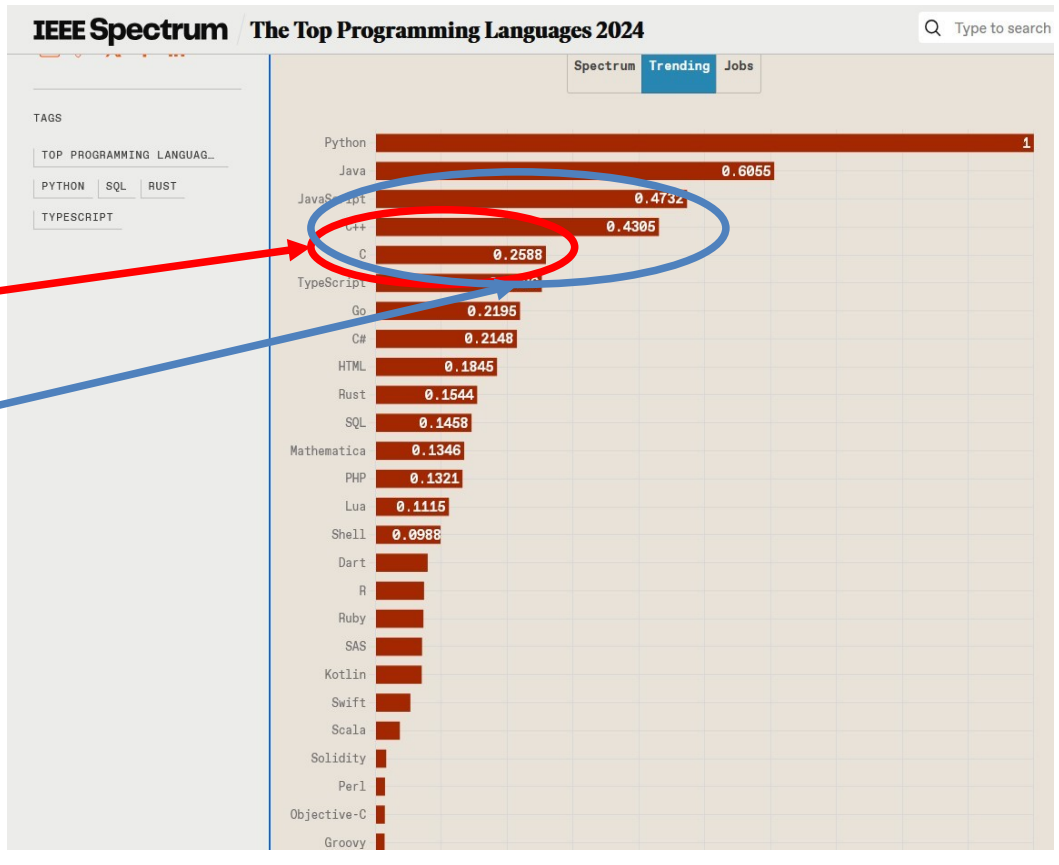
C Popularity Today





C Popularity Today





Syntax

```
int i = 0 ; // this is a comment

// a statement always ends by « ; »

float  x, y; // declaration of two variables of type  « float »

x = 0 ; // statement - variable x is assigned the float value 0 ;

y = 1/x ; // division by zero ! Will compile OK but exception will occur at run-time

/*

Alternate way of commenting - permits to delimit many lines

*/
```

Curly Brackets - braces

{ } An open brace must be closed - else Compiler error

- Delimit a sequence of code or data : function, conditional block, data structure...
- Data declared inside a block are valid ONLY WITHIN IT !

Conditional execution - if

```
bool a = false;

// some code here will modify expr...

 if (a) {
        // write some code here
     }
else {
       // write some other code here
    }
```

Conditional execution - for

```
// this code will list the first 100 integers from 0 to 99
for (int i = 0; i <= 100; i++)
{
   printf("The value of i is: %d", i);
}
```

Conditional execution - while

```
// this code will ALSO list the first 100 integers from 0 to 99
int i = 0;
while (i <= 100)
{
   printf("The value of i is: %d", i);
   i++; // equivalent to "i = i+1"
}
```

Prefixes vs Suffixes

```
int i, j;
i = 5;
i++; // increases i by one : i = 6
++i; // does the same : i = 7 (was 6)
//BUT if used in an assignment  :
i = 5;
j = ++i; // (i is 6, j is 6) because i is incremented first
i = 5;
j = i++; // (i is 6, but j is 5 as the increment took place AFTER assignment)
```

Keywords

These reserved words are case sensitive. C89 has 32 reserved words, also known as keywords : they cannot be used for any purposes other than those for which they are predefined:

auto break case char const continue default do
double else enum extern float for goto if int long
register return short signed sizeof static struct switch
typedef union unsigned void volatile while

Operators

Symbols used to specify the manipulations to be performed:

Arithmetic: +, -, *, /, %

Assignment : =

augmented assignment: +=, -=, *=, /=, %=, &=, |=, ^=, <<=, >>=

bitwise logic: ~ , & , | , ^

bitwise shifts: “ , >>

Boolean logic: ! , && , ||

conditional evaluation: ? :

equality testing: == , !=

calling functions: ( )

increment and decrement: ++, --

member selection: . , ->

object size: sizeof

type: typeof

order relations: < , <= , > , >=

reference and dereference: & , * , [ ]

sequencing: ,

subexpression grouping: ( )

type conversion: (typename)

« Hello World »

```c
#include <stdio.h>

int main()
{
    printf(" \n Hello World! \n");
}
```

« Hello World »

```
#include <stdio.h>

int main()
{
   printf(" \n Hello World! \n");
}
```

Include file referencing standard library for inputs/outputs functions/subroutines

« Hello World »

```
#include <stdio.h>

int main()
{
   printf(" \n Hello World! \n");
}
```

The Program Entry Point : a function named « main » (reserved word)

« Hello World »

```
#include <stdio.h>

int main()
{
    printf(" \n Hello World! \n");
}
```

Braces or Curly Brackets : delimiter for any piece of code and/or data

« Hello World »

```
#include <stdio.h>

int main()
{
    printf(" \n Hello World! \n");
}
```

Calls a stdio function to output a message to the standard output display

Functions

Function (procedure, method, subroutine, routine, or subprogram) :

- callable unit of software logic that has a well-defined interface and behavior and can be invoked multiple times.
- Function parameters are passed by value
- Arrays are passed as pointers, i.e. the address of the first item in the array.

Function call

```
float SquareFunction(float x)
{
    float y;
    y = x * x;
    return y;
}
```

```
float SquareFunction(float x) ; // function's « signature »
int main()
{
    float x, y;
    printf("Enter a float number: ");
    scanf_s("%f", &x);
    printf("you entered %f", x);
    y = SquareFunction(x);
    printf("\n the square of this number is : % f", y);
    return 0;
}
```

Data Types

Type	Memory size	Values Range
char	1 byte	0 to 255
int	4 bytes	-2,147,483,648 to 2,147,483,647
float	4 bytes	1.2E-38 to 3.4E+38
double	2 or 4 bytes	2.3E-308 to 1.7E+308
bool	2 or 4 bytes	0 or 1
short	2 bytes	-32,768 to 32,767

Pointers

C supports the use of pointers, a data type that contains the address of an object or function in memory.

- Careless use of pointers is potentially dangerous.
- Pointers can be manipulated using assignment or pointer arithmetic.
- At run-time a pointer value is typically a raw memory address
- since a pointer's type includes the type of the thing pointed to, expressions including pointers can be type-checked at compile time.

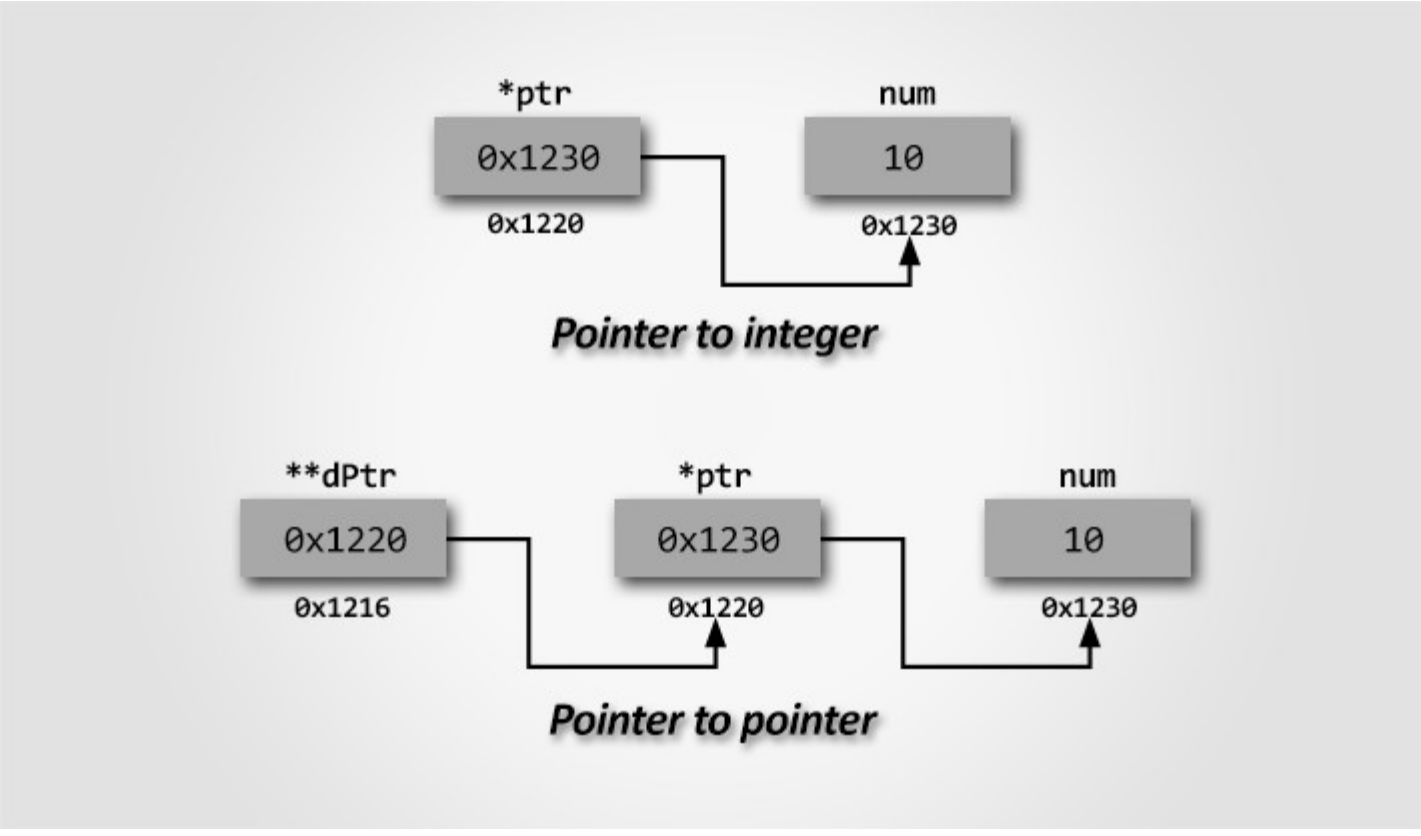
Pointers – operators « * » and « & »

```
int* ptra; // declares a pointer ptra to an integer

int a; // declares an integer a

a = 5; // the value of a is now 5

ptra = &a; // assigns to ptra the value of the address
of a

int b = *ptra; // b is assigned as value the content

               // of memory address pointed by ptra,
i.e. 5
```

Pointers

A variable that stores a memory address.

- efficient handling arrays and structures.
- Useful to pass & return multiple values from a function.
- allow dynamic memory allocation
- Must be practiced well, not for beginners



Pointers

```
// add two number using pointers
int num1, num2, sum;
 int* ptr1, * ptr2;
 ptr1 = &num1; // contains address of num1
 ptr2 = &num2; // contains address of num2
 printf("Enter any two numbers: ");
 scanf_s("%d%d", ptr1, ptr2);
 sum = *ptr1 + *ptr2;
 printf("Sum = %d", sum);
```

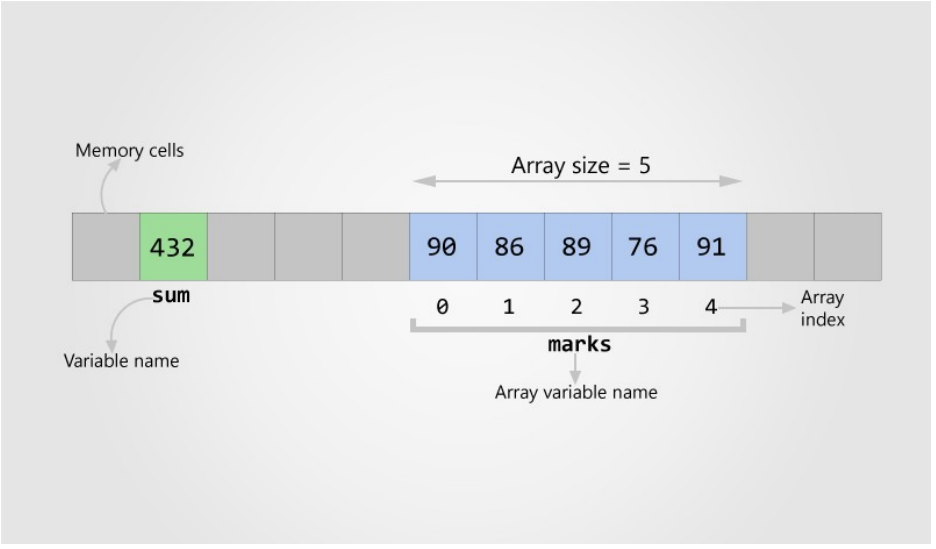
In this code we don't use « & » (address of) operator in scanf() :

scanf() takes the actual memory address where the user input is to be stored. The pointer variables ptr1 and ptr2 contain the actual memory addresses of num1 and num2. Therefore we need to NOT prefix the & operator in scanf() function in case of pointers.

Arrays

Data structure holding **finite sequential** collection of **homogeneous** data.

- **finite** – nb of data is finite, fixed prior to use.
- **sequential** in memory → can be indexed.
- **homogeneous** – The data are of the same type.
- **One-dimensional** or **Multi-dimensional** array



Arrays

```
int marks[5]; // defines a one-dimensional array of 5 integers

for (int i =0 ; i<5 ; i++)

{

marks[i] = 0 ;// init array via a loop over array index

}
```

Arrays

```
BUG : where is it
int marks[5]; // defines a one-dimensional array of 5 integers
for (int i =0 ; i<=5 ; i++)
{
marks[i] = 0 ;// init array via a loop over array index
}
```

Arrays

```
BUG : where is it
int marks[5]; // defines a one-dimensional array of 5 integers
for (int i =0 ; i<=5 ; i++) // write beyond marks array
{
marks[i] = 0 ;// init array via a loop over array index
}
```

Arrays

Array types in C are traditionally of a fixed, static size specified at compile time. The more recent C99 standard also allows a form of variable-length arrays. However, it is also possible to allocate a block of memory (of arbitrary size) at run-time, using the standard library's malloc function, and treat it as an array.

Memory Management

C provides three principal ways to allocate memory for objects:

- **Static memory allocation**: space for the object is provided in the binary at compile-time; such object « exists » as long as the program is loaded into memory.
- **Automatic memory allocation**: temporary objects can be stored on the stack, and this space is automatically freed and reusable after the block in which they are declared is exited.
- **Dynamic memory allocation**: blocks of memory of arbitrary size can be requested at run-time using library functions such as malloc from a region of memory called the heap; these blocks persist until subsequently freed for reuse by calling the library function realloc or free

Arrays – Dynamic Memory Allocation

```
int marks[10]; // array length is 10. What if we need to change it
```

A runtime case needs only 6 elements → 4 indices are wasting memory. We need to decrease length from 10 to 5.

If on contrary there is a need for 3 more elements, 3 indices more are required. The length needs to increase from 10 to 13.

This is possible thanks to **Dynamic Memory Allocation** in C : the size of an Array can be changed at runtime. 4 functions provided by C under <stdlib.h> are :

Malloc(), calloc(), free(), realloc()

Arrays – Dynamic Memory Allocation

```
ptr = (int*) malloc(100 * sizeof(int)); //allocates 100 int

ptr = (float*) calloc(25, sizeof(float));//contiguous alloc

free(ptr);// de-allocates the memory.

ptr = realloc(ptr, newSize);// changes memory allocation
```

Structures

A way to design custom data type.

```
// Student type definition
struct student
{
   char name[40];   // Student name
   int  age;        // Student age
   int mobile;                                   // Student mobile nb
};
```

Structures

```
struct student student1; // Simple structure variable

struct student* student2;   // Pointer to student

student1.age = 26;          // Assign age of student1

// Init pointer to student through dynamic memory allocation

student2 = (struct student*)malloc(sizeof(struct student));

student2->age = 29 ;       // Student2 is a pointer to student
```

Structure (C) vs Class (C++)

Structure : bundle of data – no methods, no action

- Members are public.
- Declaration via struct keyword.
- used for grouping different datatypes.

```
struct structure_name {
structure_member1;
structure_member2;
};
```

Class : a class **does** things.

- Members are private by default.
- declared using the class keyword.
- used for abstraction, inheritance.

```
class class_name {
data_member;
member_function;
};
```

IDE - prerequisites

What is needed :

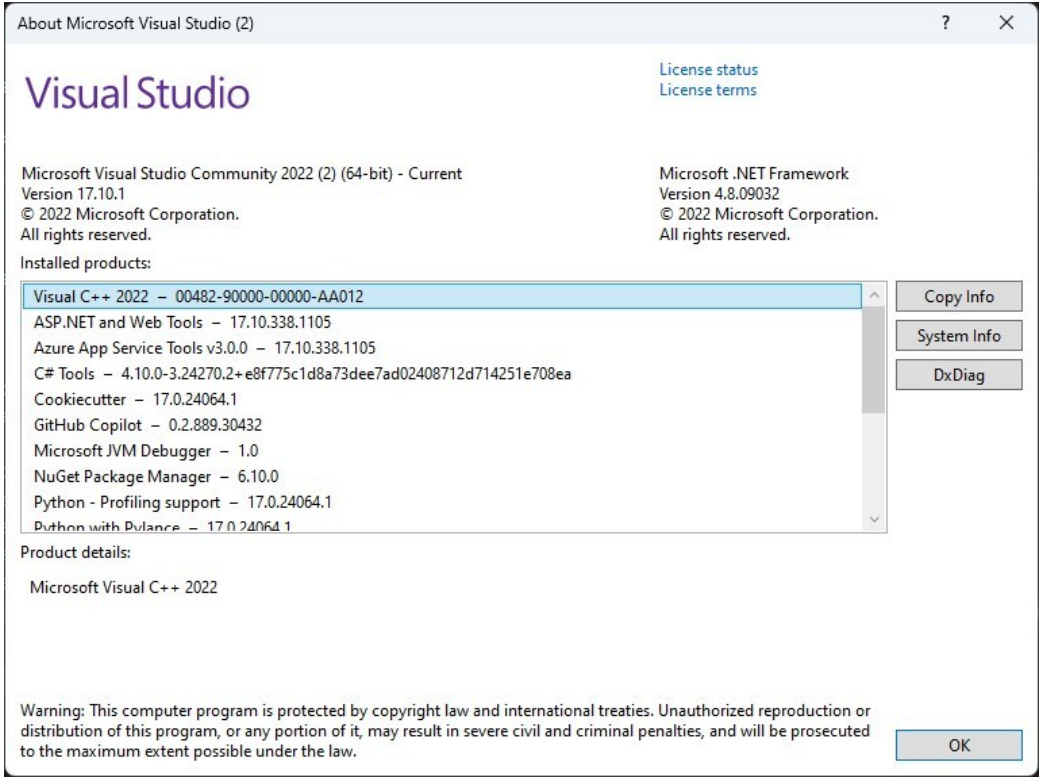
- A recent PC (Windows) with ~20Gb Disk Space

- Visual Studio Community 2022 : our IDE (Integrated development environment) https://visualstudio.microsoft.com/fr/vs/community/

Visual Studio

Once installed,

Help→About shows :



Session 1 – Hands-on exercises (1/4)

« Programming is learned by writing programs. »

Brian Kernighan (one of the « C » fathers)

“Experience is simply the name we give to our mistakes.”

Oscar Wilde

Session 1 – Hands-on exercise (2/4)

Exercise 1

- Install Microsoft Visual Studio Community 2022
- Create a folder « AdvancedC » for the course with a subfolder « Source »
- MAKE SURE YOUR ANTIVIRUS IS DEACTIVATED FOR THESE FOLDERS
- Start Visual Studio
- Choose « Create a new project » and choose Console
- Code the Hello World program, build it and execute it

Session 1 – Hands-on exercise (3/4)

Exercise 2

Create a program that displays the first 100 integers (Hint : use « for » or « while » loop with proper initialization)

Session 1 – Hands-on exercise (4/4)

Exercise 3

- Create a program that
  - gets a **x** number from user
  - calculates its square, and displays it

  For that :
  - Code a function **SquareFn** (coded above the main code) - hint : **float y = x*x ;**
  - Code main function code below to get a float x from the user (hint : `scanf_s("%f", &x);`)
  - Call function **SquareFn with x as input argument (hint : float y = SquareFn(x);)**
  - display the result - Hint : **printf("\n the square of this number is : % f", y);**
- Modify this program so that it is composed of 2 source files, main.c for main code, functions.c for **SquareFn**